TreadOps A Governed Platform for Tyre Inspection with MLOps and LLMOps

Arav Saxena Mahi Laddha
Department of Artificial Intelligence and Machine Learning
Symbiosis Institute of Technology
Symbiosis International Deemed University Pune India

Abstract—Visual inspection of tyre casings is a useful setting for studying the gap between a trained classifier and an accountable machine learning service. TreadOps combines image classification, conservative inspection routing, reproducible model development, controlled deployment, monitoring, and a tool-grounded conversational interface. This paper describes the implemented platform and analyzes experiment records preserved in its public repository. A source-aware data audit retains 3,665 unique photographs from three public datasets after removing exact duplicates and preserving related-image groups. Three completed development benchmarks compare robust ConvNeXt-Tiny, YOLO26s, and YOLO26n classifiers. ConvNeXt-Tiny is the only completed candidate to satisfy both configured development targets, with recall 0.954 and specificity 0.970. The subsequently packaged model records recall 0.911 and specificity 0.971 on 558 held-out photographs, showing that development acceptance does not establish the desired held-out recall of 0.95. Operational triage therefore separates low-score candidates, expert review, and defect flags, while reserving final clearance for a human inspector. Additional repository experiments evaluate a vision second opinion, non-tyre rejection, prompt guardrails, deployment rollback, and recovery after queue disruption. The main contribution is an auditable integration of MLOps and LLMOps rather than a new vision architecture. The results support an academic inspection prototype and identify the data, statistical, and operational evidence still required for deployment in a retreading facility.

Index Terms—tyre inspection, ConvNeXt, YOLO, MLOps, LLMOps, model governance, data leakage, drift monitoring, human review, circular economy.

I. INTRODUCTION

Tyre inspection connects a visible surface condition with a decision whose consequences extend beyond image classification. A photograph may show cracked rubber, a cut, uneven wear, an embedded object, or a surface that appears intact. An intake operator must decide whether to gather further evidence, request expert assessment, or remove a casing from the reuse stream. That decision also depends on information a photograph cannot reliably provide, including internal structural integrity, repair history, loading conditions, and the suitability of a casing for a particular retreading process. A useful software system must therefore support inspection without presenting a visual score as an engineering certificate.

TreadOps addresses this problem as an independent academic platform developed by Arav Saxena and Mahi Laddha at Symbiosis Institute of Technology, Pune [1]. The project uses publicly available photographs and explicitly simulated operational metadata. Its use of retreading as an application setting does not imply access to proprietary industrial inspection records or implementation within Michelin production systems. The system accepts photographs, selected video frames, and camera observations, computes a pinned model prediction, and records the evidence needed to explain the resulting route. A qualified inspector remains responsible for the final disposition of the casing.

A classifier alone leaves several practical questions unanswered. The operator needs to know which preprocessing contract produced an input tensor, which model version produced the score, whether that version passed its release criteria, and whether the current camera conditions resemble those used in evaluation. The engineering team needs to reproduce a result after changing a library, recover from a worker restart without duplicating an inspection, and reverse an unsuitable release without changing historical records. These requirements are system properties. They cannot be established from a single validation accuracy or a screenshot of a successful prediction.

The same distinction applies to the conversational interface. TreadOps includes Bib, a LangGraph-based copilot that can submit an inspection and retrieve model, monitoring, and circularity information. A fluent answer can still misstate a numerical result, imply that a tyre is safe, or confuse an uncertain prediction with a confirmed defect. The copilot therefore requires its own versioned prompts, deterministic guards, evidence retrieval, tool permissions, traces, and evaluation cases. These controls form the LLMOps layer around the visual inspection service, while the visual classifier retains responsibility for its numerical prediction.

The current repository also illustrates why papers must describe the checked implementation rather than the original project plan. Earlier TreadOps experiments used YOLO26 image classifiers. A later multi-source benchmark selected a robust ConvNeXt-Tiny classifier for serving, while a separate YOLO26 damage detector remained an experimental localisation aid. This paper follows that later implementation. References to YOLO in older prompts, model names, and documentation are interpreted as historical interface terminology rather than evidence that the current serving classifier belongs to that family.

We make four contributions. First, we describe a data and preprocessing contract that combines duplicate handling, grouped roles, immutable provenance, and comparable vision inputs. Second, we explain a release workflow that binds

evaluation evidence to a verified model bundle and uses generation checks for promotion and rollback. Third, we analyze a conversational workflow whose tools operate on the same inspection records as the application. Fourth, we synthesize the repository's stored results while separating development benchmarks, locked-test reporting, supporting component experiments, and operational drills. The resulting account exposes both working controls and important remaining weaknesses.

The central finding is that operational accountability improves even when predictive performance remains imperfect. ConvNeXt-Tiny satisfies the combined development gate, but its reported held-out recall remains below the configured target. This gap makes conservative routing, expert review, and explicit failure evidence essential parts of the platform. It also limits the conclusion: the project demonstrates an integrated engineering prototype, not an independently validated replacement for physical inspection.

II. RELATED WORK AND TECHNICAL FOUNDATIONS

A. Visual condition classification

Public tyre datasets make supervised inspection research accessible but constrain what a classifier can learn. The D1 dataset published by Pathmanaban and colleagues provides defective and good-condition tyre images captured with a mobile camera [2]. TyreNet adds photographs from service stations and showrooms with different capture conditions [3]. Their labels support a broad binary condition task. They do not establish exhaustive coverage of failure mechanisms, provide a complete physical inspection protocol, or guarantee that visually similar images represent different physical tyres. TreadOps consequently treats source documentation, archive contents, and duplicate relationships as separate evidence.

ConvNeXt modernizes convolutional network design while retaining standard convolutional building blocks [5]. TreadOps uses the Tiny variant through torchvision and replaces its classification head with two outputs. The project compares it with classification variants of Ultralytics YOLO26, a family documented for several visual tasks [6]. These architectures are established external components. The contribution of TreadOps lies in their controlled comparison and integration into a common operational contract, rather than a claim to have invented a new backbone or a new general-purpose detector.

Image classification and object detection answer different questions. The main classifier estimates whether the image belongs to the defective category. A localisation model instead predicts boxes associated with particular surface damage classes. TreadOps maintains this distinction in its authority model. A bounding box can make an inspection easier to review, but a low-quality detector must not gain decision authority simply because its overlay looks plausible. This separation becomes especially relevant when a classifier attends to global ageing texture while a small, fresh cut occupies only a few pixels.

B. Calibration and uncertainty under shift

Guo et al. show that modern neural network confidence often benefits from post-training calibration and study temperature scaling as a practical method [7]. TreadOps fits a scalar temperature on a dedicated calibration role before selecting an evaluation threshold on a different role. This ordering reduces direct reuse of the training objective for operational scoring. Calibration changes the interpretation of scores, however, and does not create information about defects missing from the training data. A calibrated low probability can still be wrong for an unfamiliar damage pattern.

Ovadia et al. examine predictive uncertainty under dataset shift and show why calibration measured under one distribution is insufficient evidence for another [8]. The TreadOps low-light experiments provide an application-specific example. A model that performs well on ordinary development images can lose substantial recall after brightness reduction. The project therefore evaluates controlled corruptions, reports source-specific outcomes, and monitors image descriptors. These checks are complementary. A drift statistic detects a change in inputs, whereas labelled recall measures a change in task performance.

C. MLOps and system maintenance

Sculley et al. describe how data dependencies, configuration, feedback loops, and changing environments create technical debt around machine learning systems [9]. TreadOps responds by assigning explicit ownership to different records. Git tracks source and configuration, DVC tracks reproducible data and artifact dependencies, MLflow tracks experiments and registered model evidence, and PostgreSQL stores authoritative jobs and deployments. These responsibilities reduce ambiguity when an operator asks which release handled an inspection or why a challenger could not be promoted.

DVC represents machine learning workflows as dependency-aware stages [10]. MLflow provides run tracking and model registry capabilities [11]. Neither tool alone defines the full semantics of a safe deployment. TreadOps adds task-specific gate checks, a bundle hash, golden-image warm-up, an expected deployment generation, and an audit reason. ONNX Runtime provides a separate execution path for exported models [12], which introduces a need to test numerical and routing parity between native and exported inference. The platform treats that parity as release evidence instead of assuming export preserves behavior automatically.

D. Tool-grounded assistants and auxiliary vision

LangGraph supports explicit stateful orchestration for agents and tool calls [13]. TreadOps uses a graph to place guards before and after planning, invoke application tools, and preserve execution traces. The graph structure helps make the workflow inspectable, but the safety behavior depends on the implemented rules, tool contracts, and evaluation cases. In this project, the language model explains application evidence and requests actions that the API independently authorizes. It

cannot override the model release pointer or convert a candidate route into final clearance.

Depth Anything V2 provides monocular depth estimation [14]. Its inclusion in TreadOps is advisory: the relative map supports framing inspection and a visual account of the nearest surface. It supplies neither calibrated millimetres nor a reliable measurement of tread depth. Similarly, retreading impact factors from Michelin's public guidance support a transparent scenario estimate [15], rather than a measured life-cycle assessment of the inspected tyres. These boundaries matter because attractive auxiliary outputs can otherwise appear more authoritative than the available evidence permits.

III. RESEARCH QUESTIONS AND EVIDENCE MODEL

A. Research questions

The first research question asks whether a common, source-aware evaluation contract can select a usable classifier without hiding trade-offs between defect recall, specificity, source performance, and computational cost. The second asks whether the surrounding platform can preserve the identity of a decision across asynchronous processing and model releases. The third asks whether a multimodal assistant can remain grounded in application records while enforcing a limited scope. A fourth question concerns operational observability: can the platform expose distribution changes and recovery behavior without interpreting every alert as proof that a model has failed?

These questions determine the evidence required by the paper. Classification claims require labelled cohorts, thresholds, and confusion counts. Release claims require records of gate checks, pointer changes, warm-up, and rollback. Copilot claims require the prompt version, execution mode, case definitions, and evaluation output. Reliability claims require a workload definition and a distinction between accepting a request and completing the corresponding job. Environmental claims require the counting rule and the source of each conversion factor. A single headline performance figure cannot answer all four questions.

B. Repository snapshot and observation boundaries

The technical account is based on repository commit 6b3a30e9612918c1ad1f2cc8d26d6836750ebbc5, inspected on 29 October 2026 [1]. Stored experiments were primarily recorded on 22–24 October 2026. No new model training, industrial trial, live-provider benchmark, or end-user study was performed for preparation of this manuscript. Numerical results are drawn from preserved JSON and JUnit files, checked against configuration and code where available. Values reconstructed from reported rates and support counts are identified as derived quantities.

The repository contains evidence from successive implementations. Its aggregated status file describes an earlier YOLO release, whereas the current metrics file and README describe the later ConvNeXt release. This paper retains that temporal distinction. It does not join an early release's model latency, a later release's accuracy, and a third configuration's second-opinion outcome into a single supposedly measured system. Tables therefore identify the applicable cohort and model. This evidence model is necessary for reproducible interpretation when a project evolves faster than its summaries.

TABLE I
EVIDENCE TYPES AND THEIR INTERPRETATION

Evidence	Population or scope	Interpretation
Development gate	Calibration plus policy roles	Acceptance evidence after development use
Locked test	Frozen candidate and held-out roles	Reported generalization on public images
Auxiliary studies	Version-2 YOLO and component cohorts	Separate from current ConvNeXt results
Operational drills	Specified local workloads	Recovery and admission evidence
Impact estimate	Verified retreads and public factors	Scenario arithmetic rather than measured emissions

C. Implemented behavior and prospective recommendations

The paper distinguishes observed code paths from recommendations for future validation. For example, deployment generation checks, pinned inspection bundles, and the three-way score policy are implemented controls. A prospective facility trial with independent physical casing labels is a recommended next experiment. Likewise, the stored second-opinion experiment evaluates YOLO26s version 2. It motivates a supporting control but does not establish the same benefit for the current ConvNeXt bundle. Maintaining that distinction prevents a plausible system design from being misreported as a completed experiment.

The analysis also distinguishes configured targets from statistical guarantees. A development recall target of 0.95 specifies an acceptance rule. Passing it on a finite cohort does not prove that population recall is at least 0.95, especially when the cohort also participates in calibration or threshold selection. Confidence intervals, source support, repeated seeds, and an independent test set all change the strength of the conclusion. The results are interpreted with these limitations throughout the paper.

IV. PLATFORM REQUIREMENTS AND ARCHITECTURE

A. Functional requirements

The primary user flow begins with an authenticated inspector submitting an image. The platform validates the input, stores sanitised media, creates an inspection and job, pins the active bundle, and returns an acceptance response. An inference worker later produces the prediction and stage evidence. The application displays the score, route, quality findings, visual aids, and model lineage. A human reviewer can record the eventual disposition. Video and camera flows reuse this inspection contract rather than creating a separate untraceable prediction channel.

ML engineers require an eligible data snapshot, allowlisted training recipes, evaluation reports, comparative evidence, and

replay tools. Administrators require controlled promotion and rollback with an expected generation and an audit reason. Viewers can ask Bib questions about accessible system evidence, while attaching images and changing records depend on their role. This division is enforced at the application boundary. The assistant's ability to describe an action does not grant the requesting user permission to execute it.

B. Components and authority

The web application uses the Next.js App Router and a backend-for-frontend layer. FastAPI exposes the versioned service API. PostgreSQL stores users, inspections, jobs, model versions, deployments, reviews, monitoring records, and language-model traces. Celery workers perform asynchronous inference and operational jobs, with Redis supporting delivery. An S3-compatible object store holds images, model bundles, and evidence artifacts. MLflow supports experiment and model evidence. Docker Compose provides the development infrastructure described by the repository.

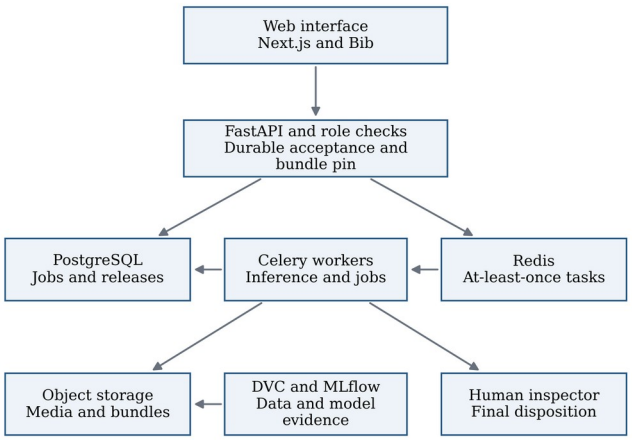


Fig. 1. Logical architecture and authority boundaries of the implemented platform. Database state remains authoritative for jobs and deployments.

The architecture gives each component a limited authority. Object storage provides bytes but does not decide the active deployment. Redis transports work but does not define the authoritative job status. MLflow records a model version but does not independently switch serving traffic. PostgreSQL holds the deployment pointer and job state. The inference worker uses the bundle pinned when the inspection was accepted. Bib invokes tools that read or submit work through these same services. This arrangement makes authority explicit enough to reason about failures and races.

C. Asynchronous processing and admission

An accepted request is not yet a completed inspection. The API can respond quickly after durable acceptance while a worker remains busy with older work. TreadOps records queue, execution, and stage timings separately so that a short HTTP response cannot mask a long backlog. The repository's sustained-load result demonstrates this distinction: acceptance latency is low, yet the queue's 95th percentile reaches tens of seconds. Capacity planning must therefore consider arrival rate, worker concurrency, expensive visual stages, and optional remote vision calls together.

The admission controller limits in-flight uploads and returns HTTP 429 with retry guidance when capacity is unavailable. This is a deliberate operational improvement over the earlier burst behavior, in which requests timed out at the client. A controlled rejection is useful because it makes overload visible and recoverable. It does not increase the capacity of the inference worker. The caller must retain its input and retry according to the API contract, while the service must avoid creating duplicate inspections for a repeated request.

D. Camera and video ingestion

For short video, the browser evaluates sampled frames for sharpness and exposure and selects a representative frame from each sector of a turn. Duplicate frames are dropped. Each retained frame creates its own pinned inspection, and the aggregate tyre verdict is conservative when any frame indicates damage. For line-camera operation, the user defines an inspection zone and the connector inspects a tyre after it stops in that zone. The same interface can display a live overlay and recent counts.

These flows improve coverage of visible surfaces, but repeated frames are not independent test examples. They may share the same casing, lighting, and background. A multi-frame system also introduces new error modes: poor sector coverage, missing surfaces, repeated triggering, and aggregation rules that increase false flags as the number of frames grows. The repository implements the user flows, while a controlled video or facility evaluation remains necessary to quantify these effects. Still-image accuracy cannot establish camera-line throughput or full-casing inspection sensitivity.

V. DATA PROVENANCE AND LEAKAGE CONTROL

A. Public sources and retained population

The original source is D1, a versioned Mendeley dataset of defective and good-condition tyres [2]. Although the publisher describes 1,854 images, the acquired archive contains 1,856 image members. TreadOps identifies 28 byte-identical copies and retains 1,828 unique photographs, comprising 1,000 defective and 828 good images. Reporting both the published description and the observed archive count avoids silently replacing the acquisition audit with a number copied from a dataset landing page. The archive receipt and checksums provide the reproducible link between the source and the local population.

Two additional datasets expand capture conditions. TyreNet supplies 1,698 archive members, of which 1,693 unique photographs are retained after exact-duplicate handling. The retained class counts are 863 defective and 830 good. The MSU sidewall and tread archive contains 1,028 image

members but shares many bytes with D1. The repository reports 883 exact matches to D1 and retains 144 additional unique photographs, including eight defective and 136 good images [1], [4]. The subtraction also implies one remaining duplicate member within that archive; the retained count is the relevant analysis population.

TABLE II
SOURCE AUDIT AND RETAINED UNIQUE PHOTOGRAPHS

Source	Members	Retained	Defective / good
D1	1,856	1,828	1,000 / 828
TyreNet	1,698	1,693	863 / 830
MSU	1,028	144	8 / 136
Combined	4,582	3,665	1,871 / 1,794

The final retained population is 3,665 unique photographs, derived as 1,828 plus 1,693 plus 144. This number is an image count, not a verified count of physical tyres. The repository groups related photographs to reduce leakage, but physical object identity is not uniformly available from the public sources. Some MSU photographs are re-shot D1 tyres and inherit the corresponding D1 group and role. Combining datasets therefore expands appearance diversity more than it expands independent physical evidence. This distinction is particularly important when interpreting source-specific confidence intervals.

B. Exact duplicates and related-image groups

Exact duplicates are detected by content hashes. Removing identical files prevents the most direct form of train-test contamination, but different photographs of the same tyre can still share the defect pattern, surrounding scene, and capture session. TreadOps also reviews candidate relationships using perceptual hashes, fixed-encoder similarity, and sequence information. Its configuration combines a small perceptual-hash distance with a cosine-similarity condition, stronger embedding similarity for likely re-photographs, and additional capture-order rules. Candidate decisions are retained in a duplicate-review artifact rather than hidden in an undocumented cleanup step.

For D1, the review produces 948 label-pure groups. The stored audit reports no shared original hashes or groups across roles. Label-pure grouping means that the grouped examples agree on the assigned class, not that every group is necessarily a confirmed physical casing. Threshold-based grouping can merge different tyres or fail to join views of the same one. The project records its grouping assumptions in an architecture decision record so that a future curator can review their effect. An independently labelled physical-identity audit would strengthen this part of the methodology.

The MSU overlap provides a concrete reason to check duplication across sources instead of processing each source in isolation. If D1 training photographs reappear under a different dataset name and are randomly assigned to a later test set, a model can appear to generalize across sources while seeing identical content. TreadOps compares new inputs with the existing corpus before role assignment. Newly discovered photographs may join a frozen group, but they cannot move an original D1 image between roles. The original image-to-role mapping is hashed and checked by a data test.

C. Role separation and actual counts

The deterministic split uses seed 42 and nominal outer fractions of 70% training, 15% validation, and 15% test. Within the outer training pool, a development-validation role selects checkpoints. The outer validation pool is split into calibration and policy roles. Temperature fitting uses calibration images, while binary threshold selection uses policy images. The test role remains reserved for final reporting on a frozen candidate. Because whole groups are assigned together and labels are stratified, actual image fractions can differ slightly from the nominal ratios.

TABLE III
IMAGE COUNTS IN EACH DATA ROLE

Role	D1	TyreNet	MSU	Total
Development train	1,084	1,006	83	2,173
Development validation	192	178	17	387
Calibration	137	128	5	270
Policy	138	127	12	277
Locked test	277	254	27	558
Total	1,828	1,693	144	3,665

Across all sources, the benchmark records 2,173 development-training images, 387 development-validation images, 270 calibration images, 277 policy images, and 558 test images. Their sum is 3,665. The calibration and policy roles together form a 547-image development gate cohort, including 280 defective and 267 good images. The test cohort contains 282 defective and 276 good photographs. The MSU test role contains only two defective examples. Its observed recall is therefore a descriptive count with extreme sampling uncertainty, not a stable source-performance estimate.

The outer-validation gate is an intentional engineering acceptance mechanism, but it is also a methodological limitation. Calibration and policy selection use that pool before gate reporting, so the gate cohort is development evidence. Architecture decision record 0008 explicitly labels it that way. A separate locked test is needed to characterize the frozen model. Repeatedly viewing test outcomes across later development cycles can still influence future choices, even if each checkpoint is evaluated once. A prospective final holdout would be preferable for a publication claiming independent generalization.

D. Input sanitation and training eligibility

The ingestion contract bounds file size and decoded pixels, enforces permitted image formats, applies EXIF orientation, and strips unnecessary metadata. A canonical JPEG representation uses a maximum long side of 1,024 pixels and a defined encoding configuration. Corrupt or unsupported inputs receive an explicit outcome instead of silently disappearing

from the denominator. This matters for both reproducibility and operations: the set of images a model accepts is part of its deployed behavior, and metadata can contain location or device information unrelated to the inspection task.

Training eligibility is also explicit. Calibration, policy, test, and replay descendants are excluded from new training snapshots. Human-reviewed inspections may become candidates for a future curated dataset, but review alone does not automatically override lineage restrictions. A replay derived from an evaluation image remains evaluation-derived even if its brightness or blur has changed. This rule prevents the feedback loop from gradually contaminating the held-out population. It also makes the cost of maintaining clean evaluation evidence visible as the operational dataset grows.

VI. VISION PROCESSING AND EXPLANATION

A. Canonical image to common tensor

The classifier receives a letterboxed 224 by 224 RGB tensor. Aspect ratio is preserved by resizing the image to fit within the target square and padding the remaining area with the configured value 114. A centre crop would risk excluding a defect near a sidewall or shoulder, so the chosen preprocessing contract avoids it. Pixel values are converted to contiguous float32 arrays in NCHW order and scaled to the interval from zero to one. The contract records the interpolation, rounding, padding, channel order, and tensor layout used by training and serving.

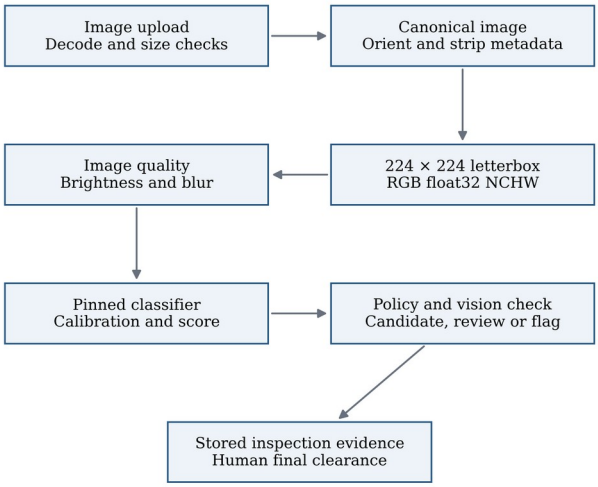


Fig. 2. Inspection path from sanitised image to stored decision evidence. Auxiliary depth and damage overlays remain outside the primary score path.

ConvNeXt requires ImageNet channel normalization. TreadOps places that transformation inside the torchvision model wrapper rather than silently changing the external tensor contract. The exported ONNX graph includes the same normalization and softmax behavior. YOLO-family and torchvision-family runners can therefore receive the same prepared input while handling their internal requirements explicitly. This design reduces the risk that a benchmark compares architectures under accidentally different crops or that serving applies normalization twice after export.

B. Image quality and stage evidence

The quality screen checks brightness and a blur descriptor. Configuration sets lower and upper brightness limits of 0.12 and 0.95 and a minimum blur-variance value of 15.0. These thresholds are engineering heuristics, not universal photographic standards. They give the application a way to identify inputs that need additional review or recapture. The policy prevents a low-quality photograph from entering the low-score candidate queue merely because the classifier assigns a small defective probability. A clearly flagged defect can still be shown while preserving the quality warning.

Each processing stage retains its timing and relevant evidence image. The interface can display the sanitised input, canonical image, letterbox, channel planes, quality result, model probability, attribution map, and annotated classification. This sequence is useful for diagnosing wrong orientation, excessive padding, or poor framing. It also prevents an explanation from becoming a purely generated narrative. The displayed stage images come from the executed pipeline, while the accompanying description explains those artifacts. Timing should be interpreted at the stage level because the complete job includes storage and explanation work beyond the model call.

C. Occlusion attribution

TreadOps uses occlusion to visualize regions that influence the classifier score. The procedure masks regions of the prepared image and measures the resulting score change. A region whose removal lowers the defective probability can be highlighted as supporting the prediction. Camera jobs use a coarser four-by-four grid, while other inspections can use an eight-by-eight grid. These settings expose a latency-resolution trade-off: a finer grid requires more evaluations and can cost much more than one forward pass.

Occlusion maps are explanatory diagnostics, not ground-truth damage segmentations. A highlighted patch can correspond to background, rubber texture, or a correlated feature rather than the defect itself. Letterbox padding and the chosen masking value can also influence the result. The paper therefore avoids claiming localization accuracy for these heatmaps. Their practical value is to help a reviewer notice that the model may be relying on a questionable region and to support targeted error analysis alongside the original photograph.

D. Relative depth and experimental damage boxes

The auxiliary depth stage uses Depth Anything V2 Small to produce a relative depth map [14]. TreadOps outlines the nearest apparent surface and reports its share of the frame as a framing aid. A single RGB image does not establish an absolute scale, and the repository's checks show that the map does not resolve fine cuts or cracks adequately for a verdict. The depth stage therefore has no decision authority. Its

estimated runtime also belongs to the auxiliary processing path rather than the classifier-only latency.

A separate YOLO26n detector predicts cracks, cuts, chunks, bulges, and foreign objects. The stored evaluation reports test mAP50 of 0.3921 against a configured minimum of 0.40. The detector consequently ships with display-only authority. Its boxes are experimental visual evidence and cannot alter the route. This is an important instance of a governance control changing product behavior: a failed model remains useful for demonstration and review, but the release package records that its predictions are insufficient for escalation authority.

VII. MODEL DEVELOPMENT AND BENCHMARK DESIGN

A. Transfer learning and robust recipes

The main comparison uses pretrained backbones with a binary classification head. Pretrained weight files are pinned by checksum in the repository, and each training record preserves the recipe and model provenance. The ConvNeXt-Tiny recipe uses AdamW, an initial learning rate of 0.0001, weight decay 0.05, label smoothing 0.1, batch size 32, a maximum of 30 epochs, and patience six. The stored run completes nine epochs and chooses the checkpoint with the lowest development-validation loss. Training uses the MPS device recorded in that experiment.

The robust YOLO26s recipe allows up to 60 epochs with batch size 16, initial learning rate 0.001, AdamW, patience 12, and seed 42. The completed run records 31 epochs. The robust nano candidate records 28 completed epochs. Comparing actual completed epochs with configured maxima matters because early stopping changes training time and checkpoint selection. The paper does not interpret maximum epoch counts as the amount of training that occurred. It also does not assume that one architecture's optimizer setting is intrinsically optimal for another.

TABLE IV
COMPLETED ROBUST TRAINING RUNS

Setting	ConvNeXt-Tiny	YOLO26s	YOLO26n
Input	224 × 224	224 × 224	224 × 224
Optimizer	AdamW	AdamW	AdamW
Completed epochs	9	31	28
Seed	42	42	42
Training time s	6,579.8	4,213.5	4,198.6
CPU p95 ms in benchmark	40.66	16.53	8.02

Robust augmentation includes horizontal flipping, small rotations, brightness, contrast, and saturation variation, along with low-light, blur, and JPEG perturbations. Vertical flipping is disabled in the listed recipe. The additional perturbations are motivated by weaknesses exposed by earlier stress tests. Their presence does not prove improved generalization by itself. A complete ablation would require the same architecture, data, and seeds with and without each augmentation. The planned non-robust YOLO26s control did not finish the benchmark sequence, so that causal comparison is unavailable.

B. Selection rule and incomplete candidates

The benchmark fixes its ranking rule before inspecting candidate scores. A candidate must first achieve defect recall at least 0.95 and specificity at least 0.90 on the development gate cohort. Among eligible candidates, the rule prefers the highest worst-source recall for sources with sufficient defective support. Near ties use low-light recall and then CPU latency. This prevents a fast model with poor clean-tyre discrimination from winning purely on recall and makes source imbalance an explicit consideration.

Five candidates were planned, but only three completed: robust ConvNeXt-Tiny, robust YOLO26s, and robust YOLO26n. The YOLO26m robust run stopped at epoch five when the benchmark was ended at the developer's request. The non-robust YOLO26s control did not start. These are incomplete experiments and have no comparable final score. They are excluded from performance ranking rather than assigned zero, estimated from a partial learning curve, or described as defeated models. The completed comparison therefore supports a bounded selection conclusion among those three runs.

C. Development benchmark and packaging are distinct

The benchmark's ConvNeXt temperature is approximately 0.581687 and its selected threshold is 0.898012. The subsequent packaged evaluation records a binary threshold of 0.836294 and correspondingly different gate recall and specificity. Although the checkpoint is adopted without retraining, changing calibration or policy artifacts can change evaluation behavior. TreadOps stores those artifacts within the release bundle. The paper consequently treats benchmark selection and packaged release evaluation as separate rows of evidence instead of averaging their metrics or describing the difference as random noise.

The adoption workflow retains the selected checkpoint's provenance while passing through the normal evaluation, export, parity, and packaging stages. This is preferable to copying a winning weight file directly into a serving folder, because a release must also contain the preprocessing contract, class mapping, calibration, policy, and evidence. A classifier's checkpoint is only one part of the executed decision. Reproducibility requires the complete bundle and the rules used to interpret its probabilities.

D. Seed sensitivity

Earlier D1-only experiments provide a useful warning about random seed variability. A second YOLO26s run with seed 43 passes the configured development gates, yet its locked-test recall is 0.887 compared with 0.953 for the seed-42 release. Paired shadow evaluation on development images also shows losses under blur and low light. The repository promotes the challenger only within a labelled release drill and then rolls back, preserving the lineage of inspections processed during the intervening generation.

This observation does not estimate the full seed distribution for either family. Two seeds are too few to quantify expected performance, and the current multi-source ConvNeXt benchmark uses one recorded seed. It does demonstrate that passing a finite development gate can coexist with substantial downstream variation. A stronger model comparison would use repeated seeds, grouped uncertainty estimates, and a separately maintained final cohort. The present results justify cautious engineering selection, while leaving claims of architecture superiority beyond the observed setting unresolved.

VIII. CALIBRATION AND CONSERVATIVE ROUTING

A. Temperature fitting and metric threshold

Let z(x) denote the model logits for image x and let T be a positive fitted temperature. The defective probability is the defective-class component of softmax applied to z(x) divided by T. Temperature is fitted using the calibration role. The policy role then chooses the highest binary threshold whose empirical recall meets the configured target, which maximizes specificity among the considered thresholds. This process binds each reported binary result to a particular checkpoint, calibration artifact, and threshold.

$$p_d(x) = softmax(z(x) \ / \ T)_d \quad (1)$$

Defect recall is the fraction of labelled defective images predicted defective, while specificity is the fraction of labelled good images predicted good. Precision uses the predicted-positive denominator. These metrics answer different operational questions: recall measures missed labelled defects, specificity measures unnecessary flags among good examples, and precision depends on the class composition of the evaluation cohort. Their interpretation therefore requires the confusion matrix and support counts, especially when the source mix differs from a future intake line.

$$Recall = \frac{TP}{TP + FN} \quad Specificity = \frac{TN}{TN + FP} \quad (2)$$

B. Binary scoring and three-way triage

The binary threshold used for evaluation is separate from the operational triage boundaries. TreadOps uses a low boundary of 0.20 and a high boundary of 0.80. An accepted-quality image below the low boundary becomes a good candidate for retread intake. A score between the boundaries requires expert review. A score at or above the high boundary becomes defect flagged. Poor-quality low-score images also require expert review. None of these states gives final clearance for use or certifies retreadability.

TABLE V
OPERATIONAL POLICY AND AUTHORITY

Condition	Route	Meaning
$p < 0.20$ and quality accepted	Good candidate	Retread intake, subject to checks and human clearance
$0.20 \le p < 0.80$	Expert review	Uncertain image-level prediction
$p \ge 0.80$	Defect flagged	Visible condition requires attention
Low score and poor quality	Expert review	Low score cannot override poor input
Auxiliary disagreement or unusable answer	Expert review	Conservative escalation when configured

Separating these rules explains why a binary false negative is not automatically an operational candidate. The current packaged classifier's binary threshold is approximately 0.836, higher than the triage flag boundary of 0.80. Some scores counted as binary negatives can still trigger a defect flag, and others enter the uncertain band. The later repository summary reports four defective test photographs in the low-score candidate route out of 282 defective examples, while the binary confusion matrix contains 25 false negatives. These quantities describe different policies and must not be presented as interchangeable measures of recall.

C. Vision second opinion

When configured, a vision language model rechecks low-score candidates. A disagreement about visible damage, or an unusable answer, moves the item to expert review instead of providing automatic reassurance. In the stored YOLO26s version-2 experiment, gpt-4o-mini with vision prompt version 2 reduces defective candidate routing from three of 150 defective test images to one of 150. The trade-off is that 28 of 116 good images initially entering the candidate route move to expert review. The added median latency per checked candidate is about 2.8 seconds.

This experiment supports a limited conclusion: a separate visual model intercepted two of three candidate misses in that particular D1 test cohort. It does not establish that the current ConvNeXt release has the same residual miss count, that the vision model is independent of all classifier errors, or that every new cut will be recognized. The additional model also introduces a provider dependency, cost, response parsing, and fail-closed behavior. An operator needs a visible indication when the check is disabled, unavailable, or inconclusive.

D. Subject rejection

A binary tyre-condition classifier must assign a score even to an image that is not a tyre. TreadOps adds an unusual-image score based on fixed-encoder distance from a tyre reference subspace. A score above 0.74 triggers further consideration, but it is not sufficient on its own to reject the input. Rejection requires agreement between the unusual-image signal and a vision response indicating no tyre subject. One-sided disagreement routes the image to an expert. Without the configured vision model, unusual images receive review rather than a definitive rejection.

The stored subject-gate experiment rejects 32 of 34 non-tyre probes and sends the two unanswered probes to review. It rejects none of the 275 validation tyre images or 277 test tyre images in that experiment. These probes include generated logos, documents, charts, colours, noise, and bundled sample

photographs. They are a useful regression set but do not represent the full population of accidental or adversarial uploads. The gate should therefore be described as an evaluated prototype control with a narrow test population.

IX. REPRODUCIBLE DEVELOPMENT AND GOVERNED RELEASE

A. Division of lifecycle responsibilities

TreadOps defines lifecycle responsibilities explicitly in its DVC pipeline: DVC owns data and model bytes plus stage reproducibility, MLflow owns run detail, and SQL owns jobs and releases [1]. The principal stages are prepare, split, train, evaluate, export parity, and package. A separate damage branch prepares, trains, and evaluates the localisation model. Stage dependencies include source code and configuration, so changing a transform or policy can invalidate downstream artifacts even when the source photographs remain unchanged.

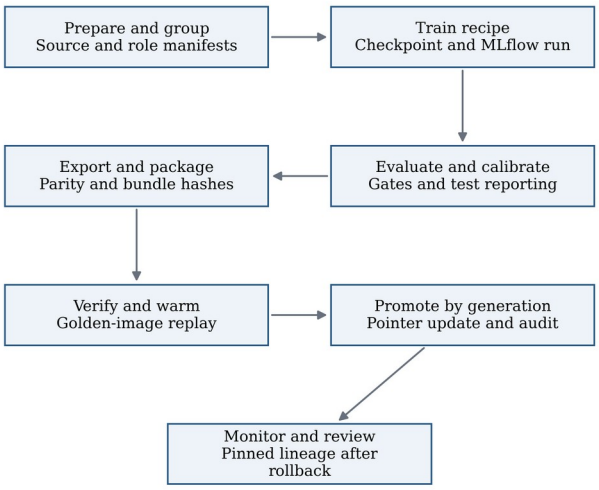


Fig. 3. Reproducible model lifecycle with verified packaging and generation-controlled promotion. Monitoring informs review and does not switch models automatically.

Git and DVC serve related but distinct purposes. Git records changes to code, small configuration, and pipeline definitions. DVC links large data and artifact versions to their dependencies and execution stages. MLflow provides experiment identifiers, parameters, metrics, and registered model metadata. The operational database records which bundle a deployment generation points to and which generation an inspection captured. A registry version can exist without being active. This distinction directly answers why data versioning, experiment tracking, and a model registry are all required in the same project.

B. Bundle contents and hard gates

A model bundle contains the executable model, preprocessing contract, class map, fitted calibration, decision policy, evaluation evidence, and supporting artifacts. Hash verification protects the association between the manifest and its files. Semantic checks confirm that the task and class map match the serving contract. Probability checks reject non-finite results. Leakage and lineage checks verify the applicable provenance requirements. A release that fails one of these hard gates cannot become active solely because a headline quality metric appears acceptable.

ONNX parity compares exported and native predictions on a specified image cohort. The configured tolerance for maximum absolute probability error is 0.001, and the mean-error tolerance is 0.0001, with a minimum comparison cohort of 50 images. Triage agreement near boundaries is also relevant because a small numerical difference can change an operational route. Earlier D1 runs compare 120 images and report maximum errors on the order of one to seven millionths. The later ConvNeXt summary reports a similarly small maximum error. These checks address export fidelity, not whether the learned classifier is correct.

C. Promotion sequence

Promotion starts by fetching and verifying the proposed bundle. An operational worker loads it independently of the serving worker and runs a golden-image warm-up. The release proceeds only after the bundle's evidence and warm-up satisfy the configured checks. The final pointer update uses a database lock and the expected deployment generation. If another release has already changed the generation, the stale request fails rather than overwriting a newer decision. The pointer change, deployment record, audit entry, and reconciliation work are coordinated transactionally.

This sequence handles a different failure class from model evaluation. A candidate can have good metrics but fail to load on the serving runtime, contain an altered file, or arrive from an administrator with an out-of-date view of the current generation. Warm-up and generation checks make those failures explicit before new traffic uses the candidate. The approach does not eliminate every race or infrastructure outage, but it narrows the set of state transitions that can change the deployed model.

D. Rollback and historical lineage

Rollback creates a new deployment generation pointing to a previously verified bundle. It does not erase the intervening generation or rewrite existing prediction records. Inspections accepted while a challenger was active keep that challenger's bundle hash even after rollback. This behavior is necessary for a later investigation: an operator can determine which model, policy, and preprocessing version actually handled an item instead of seeing only the model that happens to be active today.

The release drill in the repository promotes the seed-43 YOLO challenger and returns to the seed-42 version after reviewing comparative evidence. The drill demonstrates the transition mechanism and historical lineage. It should not be interpreted as proof that an automated monitor independently selected the correct model. The decision still depends on a human review of evidence and a release reason. TreadOps deliberately keeps drift alerts and copilot recommendations from silently changing the deployment pointer.

E. Feedback and retraining

Human review connects operational inspections with future model improvement. A reviewer can provide a label or disposition, and a later snapshot can select eligible reviewed records. The snapshot must retain the parent identity, source history, label provenance, and exclusions for evaluation-derived data. This creates a slower but more defensible feedback path than continuously training on the system's own predictions. An unverified model score is not a label, and a replayed version of a test photograph is not new training evidence.

An appropriate next release would target fresh localized damage on relatively new-looking tyres, where current models show weakness. Collecting more images of the already dominant aged-rubber pattern could improve aggregate accuracy while leaving that failure unchanged. The release workflow therefore needs an error taxonomy and subgroup evaluation in addition to the existing aggregate gates. The repository provides the technical route for new evidence to become a versioned candidate; data curation remains a separate substantive task.

X. BIB COPILOT AND LLMOPS

A. Graph execution and inspection tools

Bib executes a graph that checks input, plans, invokes tools as needed, checks output, and produces suggestions. Its tools retrieve model status, monitoring status, review queues, knowledge-base content, camera information, and circularity estimates. The image-classification tool submits an actual inspection job through the application workflow. It does not ask the conversational model to invent a probability from the attached photograph. The chat interface then displays the same stage evidence and lineage as the inspection view.

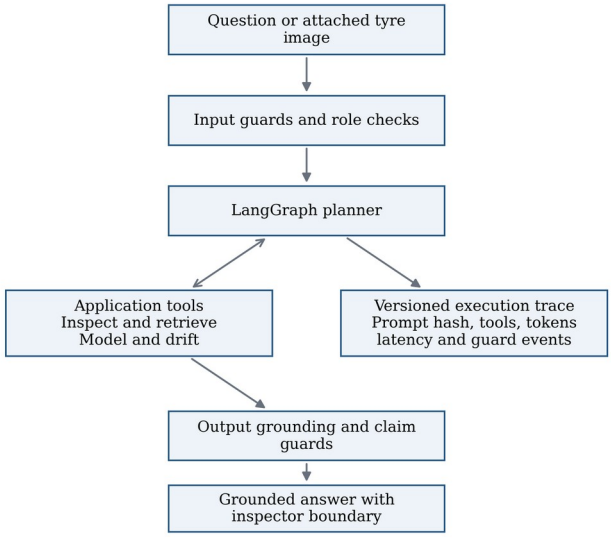


Fig. 4. Bib graph and evidence boundaries. Application tools produce authoritative records while guards constrain the generated explanation.

This design makes the assistant an interface to application evidence rather than a parallel source of truth. If a user asks whether the current model is reliable, the answer should use the stored gate and test evidence for that release. If a user asks about drift, the response should distinguish sufficient from insufficient monitoring support. Questions about physical safety require the inspector boundary. Tools can retrieve the relevant facts, while output checks constrain how the language model describes them.

B. Deterministic guards and authority limits

Input controls block known prompt-injection patterns and out-of-scope requests and redact personal information before a language-model call. Output controls check numerical grounding, suppress leaked system instructions, rewrite certification claims, and replace answers that contradict the classifier's triage. A mandatory inspector disclaimer accompanies inspection explanations. These rules are deterministic code paths with unit tests. They complement the system prompt because an instruction written in natural language alone cannot enforce every required output property.

The controls are intentionally limited. Pattern-based injection defenses can miss paraphrases and new attack forms. Numeric checks can confirm that a reported value appears in tool output while missing a wrong interpretation of its denominator or date. A statement can avoid forbidden safety wording and still imply excessive confidence. The saved copilot evaluation contains an illustrative temporal error: a queue answer describes an October date as more than three months old in an October run. Its structural checks pass, showing that numerical grounding does not ensure temporal reasoning correctness.

C. Prompt identity and tracing

Prompts are versioned files with a SHA-256 recorded for each turn. Traces preserve spans, tool activity, latency, token usage, guardrail events, feedback, and provider details. PostgreSQL stores these records next to the business data, and the offline evaluation logs to MLflow. This makes it possible to relate a user-visible answer to the prompt and model configuration that produced it. The vision second opinion also records its own model and prompt identity so that the classifier and auxiliary model do not collapse into one untraceable result.

Cost reporting is restricted to configured models with known prices. Token usage and price assumptions should remain separate because a provider may change its accounting rules or pricing. A missing price does not mean a call was free. Similarly, a trace with zero language-model tokens may represent a blocked request or deterministic execution rather than an unusually efficient generated answer. The stored execution mode is essential for interpreting performance and comparing evaluations across configurations.

D. Fallback behavior and evaluation

When an LLM endpoint is unconfigured or a call fails, Bib can use a deterministic tool-grounded fallback. The trace

labels this mode explicitly. This keeps core questions and application evidence accessible during provider problems, but its answers and routing behavior are not equivalent to the language-model path. An evaluation of the fallback cannot establish the quality of a deployed LLM, and a successful LLM evaluation cannot establish fallback coverage. The repository retains evaluations from both modes rather than concealing the distinction behind a single pass rate.

The latest stored 15-case golden-set run uses prompt version 6 and an LLM execution mode. All 15 cases pass the encoded checks. The set includes five guardrail cases, three tool cases, one circularity case, one knowledge case, four multimodal cases, and one small-talk case. Its median latency is about 5.16 seconds and its 95th percentile about 10.12 seconds. These are provider-dependent case timings for that run. They do not describe classifier latency, an external benchmark, or the expected tail latency of every future conversation.

TABLE VI
LATEST STORED COPILOT GOLDEN SET

Case category	Cases	Passed
Guardrails	5	5
Tools	3	3
Circularity	1	1
Knowledge	1	1
Multimodal inspection	4	4
Small talk	1	1
Total	15	15

E. Evaluation scope beyond a golden set

A 15-case suite is useful for release regression because it catches a known failure when a prompt or tool changes. It is too small to estimate broad safety, hallucination, or attack-resistance rates. Some cases also share the same rule or application state, reducing the effective diversity of the evidence. The observed pass rate should therefore be reported as 15 of 15 predefined cases, with the checks described. A claim of perfect copilot accuracy would imply a population and outcome definition that this experiment does not provide.

The next evaluation layer should test conflicting evidence, stale model summaries, unavailable providers, malformed tool results, changed roles, long conversations, paraphrased attacks, and requests that combine a legitimate task with an unsupported action. Human assessment should examine whether the response communicates the correct source, cohort, uncertainty, and action boundary. These tests are recommendations rather than completed project outcomes. They follow directly from the gap between structural grounding checks and the semantic error already visible in a saved answer.

XI. MONITORING AND CONTROLLED REPLAY

A. What the system monitors

TreadOps records image descriptors, model scores, source metadata, label coverage, and fixed-encoder embeddings for monitoring. Brightness and blur can change when a camera moves or lighting deteriorates. Score distributions can change because the population changes or because a model reacts differently to new inputs. Embeddings provide a broader representation of visual differences. None of these signals alone measures defect recall without reference labels. The monitor therefore presents distribution evidence as an investigation signal and separately reports whether sufficient reviewed outcomes are available for performance estimates.

Population Stability Index compares the reference and current proportions within fixed bins. The bins are derived from the reference cohort, with open outer edges so every current value can be assigned. A small smoothing value avoids undefined logarithms when a bin is empty. Freezing bins is important: recomputing quantiles independently on current data could disguise a shift by continually redefining what counts as normal. The implementation also normalizes the smoothed proportions before summing the statistic.

$$PSI = \Sigma_i \left(c_i - r_i\right) \ln\left(c_i / r_i\right) \qquad (3)$$

B. Support and alert persistence

The configured monitoring policy requires at least 200 distinct parent tyres and two consecutive windows above a candidate descriptor PSI threshold of 0.20 before opening an alert. Distinct-parent support prevents repeated views of one item from creating a misleadingly large sample. Requiring persistence reduces reaction to a single noisy window. The value 0.20 is an engineering trigger in this project, not a universal statistical decision boundary. It should be calibrated against the facility's actual variability before operational use.

An embedding domain classifier provides another diagnostic. A logistic regression distinguishes reference embeddings from current embeddings using five-fold held-out predictions. An AUC near 0.5 indicates weak separability in this representation, while a larger value suggests a distribution difference. High AUC does not tell the operator whether the difference improves or harms inspection accuracy. A new background, camera crop, or tyre style can all make the domains separable. The implementation explicitly labels this result as diagnostic rather than a recall estimate.

C. Replay isolation and scenarios

Replay creates controlled operational scenarios from eligible development images and simulated metadata. The scenarios include low light, blur, prior shift, and burst traffic. Seeds make a run repeatable, while parent linkage identifies which original image produced each event. Replays are excluded from future training eligibility and do not silently alter reference cohorts. This isolation lets engineers test monitoring and recovery behavior without making a simulated event indistinguishable from a genuine production inspection.

The stored low-light replay uses 800 events and simulated operational metadata. Brightness PSI rises approximately from 0.035 to 0.27, 0.93, and 2.07 across successive windows. Alerts open after the configured persistence requirement and duplicate alerts are merged. Mean defective probability declines as images darken, consistent with the earlier observed low-light weakness. The replay demonstrates response to a deliberately introduced covariate change. It does not estimate the frequency of that change in a real factory or prove that every future brightness shift will produce the same performance loss.

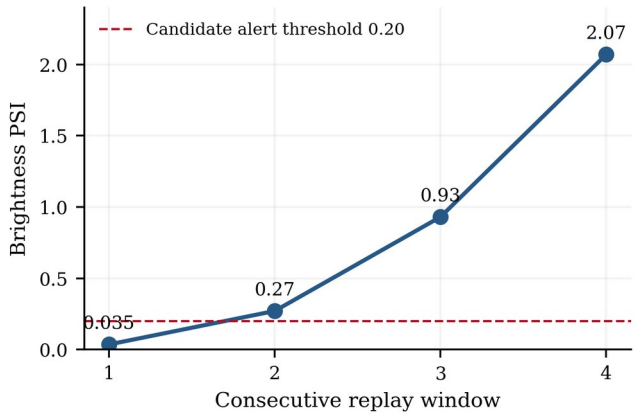


Fig. 5. Stored low-light replay brightness PSI across consecutive windows. Values are approximate repository-reported observations from simulated operational metadata.

D. Monitoring response

The useful response to a drift alert begins with diagnosis. An operator can inspect affected source and camera slices, compare sample photographs, check quality descriptors, and request additional human labels. A lighting correction or recapture may be appropriate before model retraining. If reviewed outcomes show a relevant performance loss, engineers can construct a curated snapshot and evaluate a challenger. The release still passes through the normal evidence and generation-controlled promotion path. Monitoring does not automatically select a replacement model or certify that rollback is the correct action.

This separation avoids an unstable feedback loop. Automatically retraining on every unlabelled distribution change can absorb noise, learn from the model's own mistakes, or trigger release churn. TreadOps instead uses alerts to direct human investigation and controlled experiments. The stored evidence supports the alert mechanics and replay visibility. Longitudinal evidence of alert precision, time to diagnosis, corrective-action success, and post-release recovery remains a future operational requirement.

XII. EXPERIMENTAL PROTOCOL

A. Cohorts and outcome definitions

The paper analyzes three classes of predictive experiment. D1-only baseline and challenger runs use the frozen original roles. The multi-source benchmark compares three completed robust candidates on the same 547-image development gate cohort. The final packaged metrics describe the adopted ConvNeXt release on the 558-image locked test cohort. Supporting experiments evaluate the version-2 YOLO second opinion, the subject gate, and the damage locator. Each result is attributed to its corresponding model, policy, and image population.

For binary classification, defective is the positive class. Recall, specificity, precision, and the confusion matrix are reported where recoverable. Average precision summarizes ranking behavior across thresholds, while the Brier score measures probability error. Neither average precision nor accuracy replaces recall at the chosen operating threshold. For operational triage, the relevant outcomes include defective images entering the candidate route, good images sent to expert review, and the coverage of each route. These require a separate denominator and cannot be inferred from binary scores alone.

B. Corruptions, timing, and confidence

Low-light stress reduces brightness to 45% of its original level. Blur stress applies the repository's controlled camera-blur transform. The perturbations retain the original labels as an experimental assumption. That assumption is reasonable only while the image remains interpretable; an extreme transform can remove the visual evidence needed even by a human. Stress results should therefore describe sensitivity to specified corruptions, not stand in for a representative independent camera dataset.

Timing also needs a defined boundary. The multi-source benchmark records CPU model-call timings for its completed candidates. The current metrics file separately reports a much higher model p95 value for its evaluation context, while README serving observations describe a warm ONNX path. These measurements are not directly interchangeable because runtime, warm-up, instrumentation, and auxiliary work differ or are incompletely documented. The paper retains the discrepancy instead of selecting the most attractive value. Sustained-load acceptance and queue timings are reported separately from all of them.

Confidence intervals stored with some experiments use binomial-style estimates for rates and paired bounds for comparative checks. Because related images are grouped but metrics remain image-based, image-level intervals can understate uncertainty about independent physical casings. A future analysis should bootstrap groups or evaluate one predeclared outcome per casing. In this manuscript, the observed support and raw counts carry more weight than an assertion that a threshold pass establishes a population guarantee.

C. Evidence provenance and reproducibility

The principal numerical sources are the benchmark JSON, the current metrics JSON, earlier gate and test artifacts, the second-opinion evaluation, the subject-gate record, the damage-locator evaluation, load and restart reports, copilot evaluation files, and JUnit test outputs. The repository's test-

access log records purposes and checkpoint identities for locked-test access. The paper does not claim that all original model and data bytes were re-downloaded and rerun during manuscript preparation. A full independent reproduction would require the pinned weights, raw datasets, environment, and available artifact storage.

This distinction is relevant to test reporting. The saved JUnit files contain 116 unit tests, ten data/ML tests, and 39 integration tests with no failures in those recorded runs. These are historical repository results, not tests executed by the manuscript authors on a new environment during writing. They establish that the project retains concrete verification artifacts. Their scope remains bounded by the implemented assertions and the configurations used when the files were produced.

XIII. PREDICTIVE RESULTS

A. Completed multi-source benchmark

The robust ConvNeXt-Tiny run attains development recall 0.9536 and specificity 0.9700. Robust YOLO26s reaches recall 0.9464 and specificity 0.9139, missing the recall target. Robust YOLO26n reaches recall 0.9679 but specificity 0.7715, missing the specificity target. ConvNeXt is therefore the only completed candidate eligible under the fixed combined rule. This is a stronger conclusion than choosing the largest recall alone, because the nano model's high recall comes with many more false positive flags among good images.

TABLE VII
MULTI-SOURCE DEVELOPMENT BENCHMARK

Candidate	Recall	Specificity	Combined gate
ConvNeXt-Tiny robust	0.9536	0.9700	Passed
YOLO26s robust	0.9464	0.9139	Failed recall
YOLO26n robust	0.9679	0.7715	Failed specificity

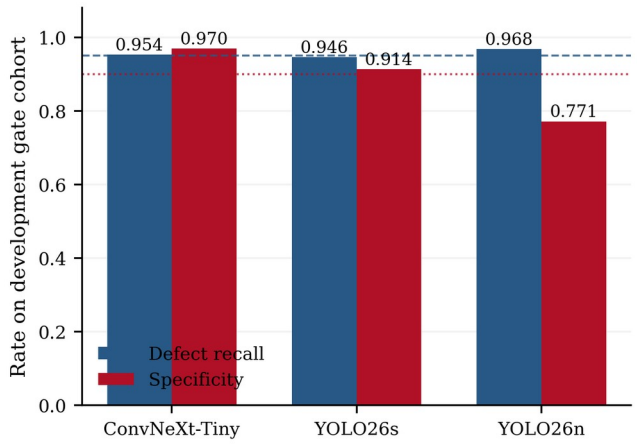


Fig. 6. Completed robust development benchmark on 547 photographs. Dashed and dotted lines show recall and specificity targets respectively.

The source slices further qualify selection. At the benchmark threshold, ConvNeXt recall is 0.940 on D1 and 0.969 on TyreNet. YOLO26s records 0.927 and 0.977, while YOLO26n records 0.947 and 0.992. Thus, the overall combined gate does not mean every source independently meets the 0.95 recall goal. The rule uses worst-source recall to rank eligible candidates after the aggregate gate, rather than imposing a separate 0.95 constraint on every source. This distinction should remain visible when presenting the chosen model.

B. Packaged ConvNeXt evaluation

The packaged evaluation records temperature 0.581687 and binary threshold 0.836294. Its development gate recall is 0.9643 and specificity 0.9663. On the 558-image test cohort, recall is 0.9113 and specificity 0.9710. With the stored support counts, these rates reconstruct 257 true positives, 25 false negatives, 268 true negatives, and eight false positives. Derived accuracy is 525 divided by 558, or 0.9409, and derived precision is 257 divided by 265, or 0.9698. The reconstructed counts are consistent with the reported rates.

TABLE VIII
PACKAGED CONVNEXT METRICS AND DERIVED COUNTS

Measure	Gate development	Locked test
Images	547	558
Defect recall	0.9643	0.9113
Specificity	0.9663	0.9710
Average precision	0.9925	0.9939
Brier score	Not used here	0.0341
TP / FN derived	270 / 10	257 / 25
TN / FP derived	258 / 9	268 / 8

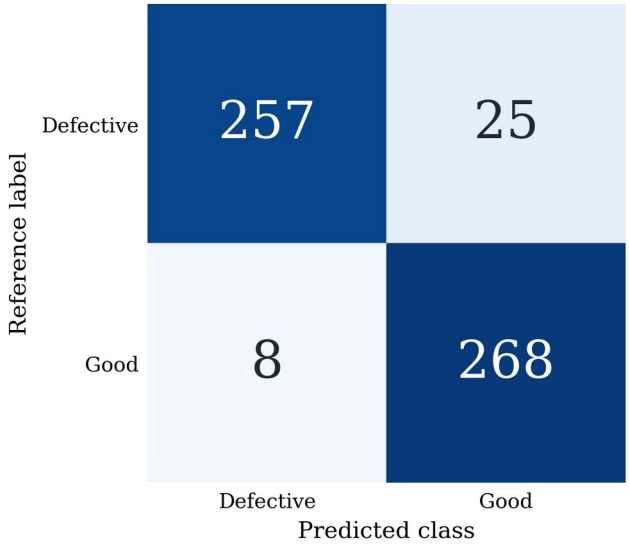


Fig. 7. Packaged ConvNeXt locked-test confusion matrix reconstructed from the reported rates and class supports. Binary threshold is 0.836294.

The central limitation is the test recall gap. A development gate pass at 0.9643 does not transfer to the desired 0.95 recall on held-out photographs. Source-specific recall is 0.880 on D1 and 0.9615 on TyreNet. The two defective MSU test images

are both missed at the binary threshold, which produces an observed zero for that tiny slice. That result is too weakly supported to characterize the entire MSU source, but it should not be hidden. The overall average precision of 0.9939 indicates strong ranking, yet it does not remove the operating-threshold misses.

C. Robustness and model-family trade-offs

The completed development benchmark reports low-light recall of 0.9536 for ConvNeXt, 0.9143 for YOLO26s, and 0.9643 for YOLO26n. Under controlled blur, the corresponding values are 0.9286, 0.9286, and 0.9679. The nano model's favorable recall under corruption must be interpreted alongside its low specificity. ConvNeXt improves the combined trade-off among the completed candidates, while none of these results establishes universal robustness. The later repository summary reports current test low-light recall near 0.901 and blur recall near 0.911, both below the configured recall goal.

Earlier D1-only evidence supplies historical context without forming a directly controlled comparison to the expanded benchmark. YOLO26s version 2 achieves locked-test recall 0.953 and specificity 0.937 on D1, but its low-light recall falls to 0.760. ConvNeXt's later D1 test recall at its own packaged threshold is lower, at 0.880. The engineering choice therefore exchanges some original-source threshold performance for better multi-source and corruption behavior. A deployment decision needs this trade-off and the associated review workload, rather than the claim that every metric improves.

D. Supporting component results

The YOLO26s second-opinion experiment changes the defective candidate count from three to one among 150 defective test images and escalates 28 of 116 good candidates. This is useful error interception at a substantial review and latency cost. The subject gate rejects 32 of 34 artificial or bundled non-tyre probes while retaining all tyre images in its validation and test cohorts. These are separate experiments on an earlier serving version. Their measured effects should be reevaluated on the adopted model and new data before being combined into a current system claim.

The damage locator's test mAP50 is 0.3921 and mAP50–95 is 0.2115. Class AP50 is high for bulges at 0.8144 and foreign objects at 0.7124, but low for cuts at 0.1938, chunks at 0.1233, and cracks at 0.1165. A single overall threshold cannot make those class differences disappear. Display-only authority is therefore consistent with the evidence. The detector is informative for some visual examples but cannot currently justify a reliable general localization or escalation claim.

TABLE IX
SUPPORTING STUDIES AND THEIR LIMITS

Component	Observed result	Scope
Vision second opinion	Defective candidates 3 to 1 of 150	YOLO v2 D1 test, 28 good candidates escalated
Subject gate	32 of 34 probes rejected	No tyre rejection in that validation/test experiment
Damage locator	Test mAP50 0.3921	Failed 0.40 gate, display-only authority
Current triage summary	4 of 282 defective in candidate route	Different policy from binary recall

E. Interpreting triage evidence

The repository's later summary places four of 282 defective test photographs in the low-score candidate route, 17 in expert review, and 261 in the defect-flagged route. The candidate fraction is a derived 1.42% of labelled defective images. It is not the false-omission rate among all candidates because the complete candidate denominator is different. It is also not a measured safety rate after physical review. The distinction matters when an audience interprets an apparently small candidate-miss fraction as proof that the system can autonomously release tyres.

A complete operational evaluation should provide a two-class by three-route table, before and after the optional second opinion, with review capacity and human decision outcomes. It should also separate recapture requests, non-tyre rejection, and missing provider responses. Current evidence supports conservative routing as a design choice and reports selected outcomes. It does not establish the throughput or residual error of a complete real inspection operation. This is the appropriate boundary for the prototype's predictive results.

XIV. RELIABILITY AND SOFTWARE VERIFICATION

A. Sustained load and queue behavior

The saved load test sends two requests per second for five minutes on an Apple M5 host with CPU inference and one inference worker. All 600 requests are accepted and all 600 eventually complete within the configured drain period. Acceptance latency is approximately 52.7 ms at the median, 122.7 ms at the 95th percentile, and 176.3 ms at the 99th percentile. These results satisfy the project's acceptance-latency and eventual-completion targets for that workload and environment. They do not establish equivalent throughput for a later bundle with extra stages or a different host.

Queue p95 in the same record is about 48.1 seconds, while worker execution p95 is about 781.8 ms. The queue grows because the arrival process can outpace the single worker's service capacity. An API acceptance p95 of 123 ms is therefore compatible with a user waiting much longer for a verdict. This is one of the most important operational findings in the repository. A future service-level objective should specify time to completed inspection, backlog limits, and maximum acceptable review delay in addition to HTTP acceptance.

TABLE X
RECORDED LOCAL LOAD AND RECOVERY RESULTS

Scenario	Outcome	Timing or limit
Sustained 2 requests/s for 300 s	600 accepted and completed	Acceptance p95 122.7 ms

Scenario	Outcome	Timing or limit
Same sustained run	Queue accumulated	Queue p95 48.1 s
Initial 180-request burst	30 accepted, 150 timeouts	No controlled 429
Burst after admission cap	8 accepted, 172 HTTP 429	Burst p95 248.2 ms
Restart under load	300 accepted and completed	Acceptance p95 72.5 ms
Saved test suites	116 unit, 10 ML, 39 integration	Zero failures in stored runs

B. Burst overload and admission repair

The first 180-request burst yields 30 accepted responses and 150 client read timeouts, with no controlled 429 responses. After the admission cap is added, the stored burst report records eight accepted requests and 172 immediate HTTP 429 responses, with latency p95 about 248.2 ms. The repair changes overload from a largely ambiguous timeout outcome into explicit admission refusal. It does not mean that 172 inspections failed after acceptance or that 180 items completed. Only accepted requests enter the corresponding completion analysis.

The overload report also contains empty sustained-load fields because that run exercises the burst profile rather than the five-minute workload. Those empty fields should not be interpreted as a failed rerun of the earlier sustained test. This is another reason to inspect workload metadata before comparing reports. The same JSON schema can hold different test profiles, and the relevant denominator depends on the profile actually executed.

C. Delivery semantics and restart recovery

Celery delivery is treated as at least once. Workers can receive a task again after a process interruption, so handlers must be idempotent and consult durable SQL state. Leases and heartbeats distinguish active work from abandoned work, while an outbox coordinates durable database changes with asynchronous delivery. A prediction already recorded for a job must not be duplicated simply because the broker redelivers its message. The implementation's correctness therefore depends on database constraints and state transitions as well as queue configuration.

The stored restart-under-load drill pauses inference for 30 seconds and restarts Redis while traffic continues. It accepts and eventually completes 300 of 300 requests, with acceptance p95 approximately 72.5 ms. Related drill records report no duplicate predictions, orphaned leases, or stuck outbox events. The exercise supports recovery for that interruption pattern. It does not prove exactly-once messaging, cover every crash location, or establish a formal recovery-time objective. Backup restoration into a scratch database is also recorded, with sampled bundle and media references resolvable.

D. Test artifacts and remaining verification

Saved JUnit reports contain 116 unit tests, ten data/ML tests, and 39 integration tests, all passing in those runs. Unit coverage includes preprocessing, policy boundaries, calibration metrics, monitoring, guardrails, model wrappers, subject gates, and camera logic. Data/ML tests check actual manifests and artifacts. Integration tests exercise API contracts, authorization, job recovery, release behavior, cameras, and supporting vision paths against real infrastructure. These artifacts are stronger evidence than a feature checklist but do not establish exhaustive branch or fault coverage.

Further verification should target failures at the boundary between components. Examples include object storage failing after a database transition, provider timeout during a low-score second opinion, a generation conflict during two simultaneous release requests, and a camera reconnect creating duplicate observations. Security review should examine access to images, traces, and retention actions in addition to role checks. These are recommendations derived from the architecture. The paper does not present them as completed penetration testing, production certification, or measured operational availability.

XV. CIRCULARITY ACCOUNTING

A. Pathways and verified dispositions

TreadOps associates its routes with retreading operations. A low-score candidate enters retread intake, an uncertain result requires expert casing inspection, and a defect flag can indicate the material-recovery pathway for further disposition. These are workflow suggestions, not a physical assessment of retreadability. The human review record can mark retread, repair, material recovery, or reinspection. Only inspector-verified retreads contribute to the sustainability ledger, preventing every optimistic classifier score from being counted as an environmental benefit.

This counting rule separates model output from a completed reuse event. An image may receive a favorable route but later fail nondestructive inspection or prove unsuitable because of internal damage. Counting at the route stage would inflate impact and reward the model for optimistic predictions. Linking impact to the verified disposition gives the estimate an auditable operational denominator. The remaining uncertainty lies in the conversion factors and the representativeness of the underlying retreading assumptions.

B. Estimate equations and example

The versioned sustainability configuration cites public Michelin truck and bus retreading guidance [15]. It uses an estimated 50 kg of raw material saved per retread and a factor of 2.3 kg CO2e per kg of raw material. Their product gives 115 kg CO2e per verified retread. For n verified retreads, estimated raw material saved is 50n kg and estimated avoided CO2e is 115n kg. An illustrative 25-retread scenario therefore produces 1,250 kg of material and 2,875 kg CO2e. This arithmetic is an example, not a measured project outcome.

$$M(n) = 50n \qquad E(n) = 115n \qquad (4)$$

The scope is truck and bus casings under the cited factors. Passenger tyres, different processes, energy mixes, transport distances, and allocation methods can change the result. The

ledger does not measure actual emissions at a facility, and the paper makes no claim of a completed comparative life-cycle assessment. Its contribution is to preserve the factor source, version, units, and counted disposition so that a future reviewer can replace assumptions without rewriting the inspection history.

XVI. DISCUSSION AND THREATS TO VALIDITY

A. What the evidence establishes

The first research question receives a qualified positive answer. A common data and input contract supports a transparent comparison, and the fixed combined rule selects ConvNeXt among three completed robust candidates. The held-out recall shortfall shows that selection remains development evidence. The second question is supported by implemented bundle pinning, durable job state, generation-controlled release, and stored recovery drills. These controls make a decision traceable even when the chosen model is imperfect or later replaced.

The third question is partly supported. Bib invokes real tools, records prompt identity, and passes a small predefined regression set. The saved temporal error demonstrates that structural checks leave semantic gaps. The fourth question is supported for controlled replay mechanics: descriptor shifts produce persistent alerts without automatic release changes. Real alert utility, maintenance cost, and corrective-action success remain unmeasured. Across these questions, the strongest evidence concerns explicit system state and controlled transitions rather than population-level predictive assurance.

B. Data and construct validity

Binary defective/good labels are an incomplete construct for casing inspection. They combine varied surface conditions and do not encode defect severity, location, repairability, or hidden structural damage. A model can learn old-looking texture as a shortcut for defective status and miss a fresh cut on new rubber. The repository documents such a gash example, including a low defective score and an in-distribution embedding distance. This is a reminder that distribution similarity does not guarantee that a specific failure mechanism is understood.

Source diversity also has limits. Public sources share duplicate content, and related-image grouping is based partly on heuristic similarity and manual review. Source labels may reflect different annotation practices. Simulated operational metadata cannot establish performance across real shifts in camera hardware, line speed, geography, or casing type. The MSU defective evaluation support is especially small. More images are useful only when their acquisition and labeling add the intended independent evidence.

C. Internal and statistical validity

The gate cohort participates in calibration or threshold development, so its acceptance rates are optimistic estimates of future behavior. Model-family selection across the same cohort adds another level of adaptation. The locked-test log improves visibility but cannot prevent investigators from learning about the problem across successive candidates and future releases. A publication-grade follow-up should freeze a prospective protocol and hold out newly collected physical casings until all architecture, augmentation, and threshold decisions are final.

The benchmark also lacks a completed non-robust control and repeated seeds for every family. It cannot isolate which improvement comes from architecture, augmentation, optimization, or source expansion. Timing values come from different execution contexts, with an unresolved difference between approximately 40.7 ms benchmark model p95 and approximately 469 ms in the current metrics file. Without a fully matched timing protocol, a claim of a fixed speed advantage would be premature. The appropriate result is the measured trade-off in each recorded setting.

D. External and operational validity

All load and recovery results come from a development environment on a single host. Real deployment introduces network delays, storage limits, multiple workers, camera transport problems, authentication systems, and possibly remote model providers. Depth, heatmaps, and vision checks can dominate total processing time. A capacity plan based only on the classifier forward pass would underestimate resources and backlog. A facility evaluation should measure completed inspections per casing, end-to-end latency, missed surfaces, duplicate triggers, and inspector review workload together.

Human review is a control with its own failure modes. Inspectors may over-trust a model, ignore a low-quality warning, or become overloaded by false escalations. The vision second opinion intercepts errors but also sends many clean candidates to review. A user study should evaluate decision quality, time, disagreement resolution, and the effect of explanations. Until such evidence exists, retaining a human in the workflow is an appropriate authority boundary but not a measured guarantee that all errors will be corrected.

E. Responsible interpretation of auxiliary outputs

The platform includes several outputs that can appear authoritative: calibrated probabilities, occlusion maps, boxes, depth maps, and avoided-emission estimates. Each has a distinct evidentiary scope. A probability depends on the evaluation distribution, a heatmap explains score sensitivity, a box requires detection validation, a monocular map is relative, and an impact estimate depends on factors and verified dispositions. The interface should keep those roles visible so that an operator can use them without inferring a stronger conclusion.

The same principle applies to agent answers. A response can be numerically grounded and still confuse a gate cohort with a test cohort or a threshold with an operational route. Future LLMOps evaluation should therefore check relations between facts, not only the presence of individual values. This is a practical research opportunity arising from the integrated system: a structured evidence schema can support tests of

source, denominator, time, policy, and authority within generated explanations.

XVII. FUTURE WORK

The most important next step is a defect-stratified dataset with verified physical casing identities. Collection should deliberately include fresh cuts, punctures, bulges, bead damage, tread separation, varied wear, and hard negative examples. Multiple views should remain linked to the same casing. Independent experts should annotate visible findings and final physical inspection outcomes separately. A prospective test cohort acquired after the development protocol is frozen would provide substantially stronger evidence than repeatedly expanding the same public-image evaluation pool.

Model experiments should use repeated seeds and completed controls under a matched preprocessing and timing protocol. Evaluation should report source and defect-type recall, calibration, three-route coverage, and human-review burden. The optional second opinion should be tested on the adopted ConvNeXt bundle with provider failures and cost included. The damage locator needs improved annotations and class-specific validation before receiving escalation authority. Depth should remain advisory unless a separately validated metric measurement process is introduced.

Operational work should define completion-based service objectives and evaluate the entire pipeline under realistic traffic. Replay can test candidate fixes, but a camera pilot must measure sector coverage, reconnect behavior, backlog, storage growth, and duplicate-casing events. The platform should also evaluate retention and restore procedures over longer histories. For Bib, a larger human-reviewed suite should examine temporal reasoning, stale evidence, unsupported safety implications, ambiguous tool results, and prompt attacks beyond the current rule patterns.

Finally, circularity accounting should incorporate locally verified process factors and clear system boundaries. A facility-level study could compare actual material inputs and emissions for verified retreads against the selected baseline, while preserving uncertainty and allocation assumptions. TreadOps already links a human disposition to versioned arithmetic. Extending that structure with measured process data would make the ledger more useful without treating a model prediction as a completed environmental outcome.

XVIII. CONCLUSION

TreadOps demonstrates an integrated approach to visual tyre inspection in which data lineage, evaluation, deployment, monitoring, and conversational assistance share explicit evidence boundaries. Its source audit retains 3,665 unique photographs, its completed development benchmark selects robust ConvNeXt-Tiny under combined recall and specificity gates, and its release workflow binds the executable model to calibration, policy, and verification artifacts. Durable asynchronous processing and pinned bundle identity make historical decisions inspectable across promotion and rollback.

The predictive evidence remains qualified. The packaged model's held-out recall is 0.911, below the configured 0.95 goal, despite strong specificity and ranking performance. Supporting vision checks reduce some errors in earlier experiments but add latency and review work. Bib passes a small structural regression suite while still showing semantic limitations in saved answers. These findings support the platform as an academic engineering prototype and identify concrete experiments required before operational adoption. The broader lesson is that reliable ML applications need mechanisms that expose uncertainty and preserve accountability when model metrics alone are insufficient.

APPENDIX A ARTIFACT EVIDENCE AND REPRODUCTION

A. Predictive evidence locations

The completed multi-source comparison is stored in docs/evidence/benchmark/20261024T170848.json. That record contains the per-source role counts, completed candidates, training histories, checkpoint hashes, gate confusion matrices, corruption outcomes, CPU timings, fixed ranking rule, and reasons for incomplete candidates. The current packaged summary is reports/metrics.json. The two files use different selected binary thresholds, so an independent analysis should preserve both and identify which artifact supplies each result. configs/recipes/convnext_tiny_224_robust.yaml and packages/treadops_ml/treadops_ml/tvmodel.py specify the adopted family, normalization wrapper, optimizer, and checkpoint-selection behavior.

Earlier YOLO evidence resides in docs/evidence/phase3/baseline_nano_224 and docs/evidence/phase5/challenger_small_224. The seed-43 challenger is in the adjacent challenger_small_224_seed43 directory. These records support historical comparisons and the release drill. They should not be substituted for the later multi-source model. The source acquisition receipts are under docs/evidence/data, while configs/data.yaml and configs/split.yaml define the documented source and role policies. The frozen D1 mapping in docs/evidence/data/d1_frozen_roles.json supports verification that the source expansion did not move original photographs between roles.

B. Supporting components and operational records

The version-2 vision second-opinion test analyzed here is docs/evidence/vision_check/test_20261023T162332.json. It records the bundle identity, gpt-4o-mini provider model, vision prompt hash, per-image routes, errors, and timing. The subject-gate record is docs/evidence/subject_gate/20261023T162114.json. The damage-locator evaluation is docs/evidence/damage_locator/v1_yolo26n_512_evaluation.json. Keeping these paths explicit prevents the latest conversational answer or README summary from becoming the only evidence of an auxiliary model's performance. Each

record identifies the narrower experiment to which its counts belong.

Load and recovery evidence is retained in docs/evidence/phase6. In particular, load_report.json contains the sustained workload and original burst failure, overload_report.json contains the repaired admission burst, and restart_drill_load.json contains the restart-under-load workload. The corresponding recovery and restore records describe state checks after the workload. Saved test suites are docs/evidence/tests/unit.xml, ml.xml, and integration.xml. The latest 15-case copilot result discussed in this paper is docs/evidence/llmops/copilot_eval_2026-10-23T162633.json, with the golden set defined in configs/evals/copilot_golden.jsonl.

C. Reproduction sequence and interpretation

An independent reproduction should first check out the cited commit, review the repository acquisition and licensing instructions, and obtain the pinned public datasets and pretrained weights. Checksums and source roles should be validated before training. The normal DVC sequence then prepares data, splits groups, trains an allowlisted recipe, evaluates the selected checkpoint, checks export parity, and packages the result. Environment and hardware details should be recorded anew rather than copied from the original MPS and CPU experiments. A changed library or execution provider can preserve accuracy while changing numerical or timing behavior.

Operational reproduction should use disposable infrastructure and the documented development commands, then run the appropriate test and drill profiles against that environment. A new reproduction is a new experiment and should retain its own results beside the original evidence. Acceptance latency, queue delay, worker time, and complete verdict latency must remain separate. Likewise, the LLM execution mode, prompt hash, provider identity, and optional vision configuration should be recorded for every assistant or second-opinion run. These requirements make disagreements between an independent result and the original repository interpretable rather than treating every discrepancy as an implementation error.

REFERENCES

[1] A. Saxena and M. Laddha, “TreadOps,” source code and stored experiment evidence, commit 6b3a30e9612918c1ad1f2cc8d26d6836750ebbc5, 2026. [Online]. Available: https://github.com/arav7781/TREADOPS. Accessed: Oct. 29, 2026.

[2] P. Pathmanaban et al., “Digital images of defective and good condition tyres,” Mendeley Data, ver. 1, 2023, doi: 10.17632/bn7ch8tvyp.1.

[3] V. S. Mahendrada and M. Parameswaran, “TyreNet A high-quality annotated dataset for tyre defect classification with deep learning models,” Mendeley Data, ver. 1, 2024, doi: 10.17632/32b5vfj6tc.1.

[4] J. E. Siegel et al., “Oxidized and non-oxidized tire sidewall and tread images,” Harvard Dataverse, doi: 10.7910/DVN/Z3ZYLI. Dataset identity and retained counts as recorded in [1].

[5] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, “A ConvNet for the 2020s,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition, 2022, pp. 11976–11986, doi: 10.1109/CVPR52688.2022.01167.

[6] Ultralytics, “Ultralytics YOLO26,” official model documentation. [Online]. Available: https://docs.ultralytics.com/models/yolo26/. Accessed: Oct. 29, 2026.

[7] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in Proc. 34th Int. Conf. Machine Learning, PMLR, vol. 70, 2017, pp. 1321–1330.

[8] Y. Ovadia et al., “Can you trust your model's uncertainty Evaluating predictive uncertainty under dataset shift,” in Advances in Neural Information Processing Systems, vol. 32, 2019, arXiv:1906.02530.

[9] D. Sculley et al., “Hidden technical debt in machine learning systems,” in Advances in Neural Information Processing Systems, vol. 28, 2015, pp. 2503–2511.

[10] Iterative, “Pipelines,” DVC documentation. [Online]. Available: https://dvc.org/doc/user-guide/pipelines. Accessed: Oct. 29, 2026.

[11] MLflow, “ML model registry,” official documentation. [Online]. Available: https://mlflow.org/docs/latest/ml/model-registry/. Accessed: Oct. 29, 2026.

[12] Microsoft, “ONNX Runtime documentation.” [Online]. Available: https://onnxruntime.ai/docs/. Accessed: Oct. 29, 2026.

[13] LangChain, “LangGraph overview,” official documentation. [Online]. Available: https://docs.langchain.com/oss/python/langgraph/overview. Accessed: Oct. 29, 2026.

[14] L. Yang et al., “Depth Anything V2,” 2024, arXiv:2406.09414. [Online]. Available: https://arxiv.org/abs/2406.09414.

[15] Michelin, “Why is tyre retreading a good option for my trucks,” public truck and bus retreading guidance. [Online]. Available: https://business.michelin.co.uk/help-advice/truck/tyre-basics-tyre-retreading. Factor scope and version as recorded in [1]. Accessed: Oct. 29, 2026.